

02 — Server Production Hardening

Revision: 1 **Parent:** 00_MASTER_INDEX.md **Prerequisites:** None (can run in parallel with Stage A)

B-01 [VERIFY] — SEC-1 Token Fail-Fast Is Actually Committed

Effort: Verify (S, ~5 min) **Source:** server/internal/config/config.go:179-207, DELTA_ANALYSIS §3 KI-1

What to check:

```
grep -A5 "refusing to start" server/internal/config/config.go
grep "ALLOW_INSECURE_DEV_TOKEN_SECRET" server/internal/config/
config.go
```

Expected: Server refuses to boot when HELIX_TOKEN_SECRET is unset AND HELIX_ALLOW_INSECURE_DEV_TOKEN_SECRET is NOT set. Loud WARN when dev fallback is used.

If PASS: Mark verified. **If FAIL:** Implement fail-fast before any other work.

Evidence: git show 7763c7c7 (the commit that landed SEC-1). Run go build ./server/cmd/ota-server and verify exit 0.

B-02 [AGENT] — Fix /readyz Probe to Return Actual Store Health

Effort: S (~30 min) **Source:** DELTA_ANALYSIS §2 SRV-NEW-2

Current state: The ReadyFunc closure in `server/cmd/ota-server/main.go` calls `repo.GetIdempotent(ctx, "__readyz__")` and returns true unconditionally. A PostgreSQL or S3 outage never flips readiness to NOT-READY.

Fix: 1. Read `server/cmd/ota-server/main.go`, find the ReadyFunc closure (around line 86-93). 2. Make it return the ACTUAL result of the store round-trip: `return err == nil`. 3. Add an S3/MinIO health probe when `HELIX_STORAGE_BACKEND=s3`: ping the bucket. 4. Add test: `TestReadyzReturns503WhenStoreUnreachable` in `server/internal/api/handlers_health_test.go`. 5. Verify: start server with no PostgreSQL → GET `/readyz` returns 503.

Guard: §1.1 paired mutation — inject a fault that makes the store fail → verify `/readyz` returns 503.

B-03 [AGENT] — TLS One-of-Pair Fail-Fast

Effort: S (~20 min) **Source:** DELTA_ANALYSIS §2 SRV-NEW-4

Current state: If exactly one of `HELIX_TLS_CERT`/`HELIX_TLS_KEY` is set, the server silently serves plain HTTP. No cross-field validation exists in `config.Load()`.

Fix: 1. In `server/internal/config/config.go Load()`, after parsing TLS fields, add:

```
go    if (cfg.TLSCertFile != "" && cfg.TLSKeyFile == "") ||
    (cfg.TLSCertFile == "" && cfg.TLSKeyFile != "") {        return
    Config{}, fmt.Errorf("both HELIX_TLS_CERT and HELIX_TLS_KEY must
    be set together, or neither")    } 2. Or warn loudly in main.go when
    the guard in the && check at line 119 rejects the TLS path due to one-
    missing. 3. Test: TestConfigRejectsOneOfTlsPair.
```

B-04 [AGENT] — Add TLS to Production Compose

Effort: S (~30 min) **Source:** Gap tracker G-65

Current state: `server/deploy/system.compose.yml` and `deploy/sword/compose.sword.yml` comment TLS as optional. Production deployment needs it.

Fix: 1. Add volume mount for TLS cert/key in the ota-server service. 2. Set HELIX_TLS_CERT and HELIX_TLS_KEY env vars in the compose file (pointing to mounted paths). 3. Add HELIX_TRUST_TLS_PROXY=true if the proxy handles TLS termination. 4. Update nginx/hxota-proxy.conf for HTTPS upstream if proxied. 5. Document that certs must be provisioned via https_certs.sh before compose up. 6. Validate: podman-compose config → exit 0.

B-05 [AGENT] — Rate-Limiter Production Defaults

Effort: S (~20 min) **Source:** PRODUCTION_READINESS_PLAN.md K14, Gap tracker G-21

Current state: G-21 claims “Closed” with HELIX_MAX_INFLIGHT=1000 in config.go. But system.compose.yml may not set it explicitly. The decision in A-05 settles the number.

Fix (after A-05 resolved): 1. Set HELIX_MAX_INFLIGHT in server/deploy/system.compose.yml and deploy/svord/compose.svord.yml. 2. Add HELIX_RATE_LIMIT_RPS and HELIX_AUTH_RATE_LIMIT with production-appropriate values. 3. Verify: podman-compose config shows the vars. Server starts with rate-limit active.

B-06 [AGENT] — Auth Hardening: Persistent Users + Durable Refresh Tokens

Effort: M (~3h) **Source:** PRODUCTION_READINESS_PLAN.md §2.7, OTA-036

Current state: Users stored in in-memory UserDirectory (lost on restart). Refresh tokens in in-memory refreshStore (lost on restart). Admin user seeded from env vars (plaintext comparison after bcrypt).

Fix: 1. Add users table to PostgreSQL schema (migration 11): sql
CREATE TABLE IF NOT EXISTS helix_ota.users (user_id TEXT
PRIMARY KEY, username TEXT NOT NULL UNIQUE,
password_hash TEXT NOT NULL, roles TEXT[] NOT NULL DEFAULT
'{}', created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()); 2.
Add refresh_tokens table: sql CREATE TABLE IF NOT EXISTS
helix_ota.refresh_tokens (token_id TEXT PRIMARY

```
KEY,          user_id TEXT NOT NULL REFERENCES
helix_ota.users(user_id),          expires_at TIMESTAMPTZ NOT
NULL,          used BOOLEAN NOT NULL DEFAULT FALSE,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()    ); 3. Implement
PgUserStore and PgRefreshStore in store layer. 4. Wire them in when
PostgreSQL is the backend (in-memory fallback otherwise). 5. Admin
seed: hash the password on first boot, store in users table instead of in-
memory map. 6. Test: TestAuthUsersPersistAcrossRestarts,
TestRefreshTokenRotationPersists.
```

B-07 [AGENT] — Device-Side min_current_version in UpdateAvailable

Effort: M (~2h) **Source:** PRODUCTION_READINESS_PLAN.md §2.3, OTA-037

Current state: Server enforces the floor (`handlers_client.go:63-76`) but the wire `UpdateAvailable` struct (`submodules/ota-protocol/types.go`) carries NO `MinCurrentVersion` field. The device cannot enforce locally.

Fix: 1. Add `MinCurrentVersion` string to `ota-protocol/types.go` `UpdateAvailable` struct. 2. Populate it in `server/internal/api/handlers_client.go` `handleClientUpdate()`. 3. Add to Kotlin DTO in `ota-android-agent/core/.../Dtos.kt`. 4. Add device-side enforcement: if `currentVersion < MinCurrentVersion`, reject update. 5. Bump `ota-protocol` submodule gitlink in parent repo. 6. Test: `TestUpdateAvailableCarriesMinCurrentVersion (server)`, `TestMinCurrentVersionRejected (Kotlin)`.

B-08 [AGENT] — Per-IP/Token Rate Limiting + Brute-Force Login Guard

Effort: S (~1h) **Source:** PRODUCTION_READINESS_PLAN.md §2.7

Current state: Only a global in-flight semaphore exists. No per-IP throttling, no login brute-force protection.

Fix: 1. Implement per-IP rate limiter middleware (use existing `rate_limit.go` pattern, extend with per-IP token bucket). 2. Add login brute-force guard: track failed login attempts per IP, block after `HELIX_AUTH_RATE_LIMIT` (default 5/min). 3. Add `X-RateLimit-*` headers to responses. 4. Test: `TestRateLimitPerIp`, `TestLoginBruteForceBlocked`.

Note: The `security_middleware.go` PII detection already exists. This extends rate limiting, not PII.

B-09 [AGENT] — Wire Security Submodule into Server

Effort: S (~30 min) **Source:** Gap tracker G-23

Current state: G-23 claims “Closed” with `go.mod replace + security_middleware.go`. The PII detection middleware IS wired (`security_middleware.go` exists and is in the middleware chain). Verify the wiring is complete.

Verify:

```
grep "digital.vasic.security" server/go.mod
ls server/internal/api/security_middleware.go
grep "securityMiddleware\|PII" server/internal/api/server.go
```

If already wired: No action needed. **If partial:** Complete the wiring (the Enforcer and Redactor from the security submodule should be instantiated and injected).

B-10 [VERIFY+FIX] — Migration Down() Methods

Effort: S (~30 min) **Source:** Gap tracker G-19

Current state: Migration struct HAS `DownSQL` string fields (migration 1-10 all have them). But there’s NO `func Down()` method to actually execute them. The `DownSQL` is stored but never used by `Migrate()`.

Fix (if operator wants runnable down-migrations): 1. Add `MigrateDown(ctx, targetVersion)` method to `PostgresRepository`. 2. Execute `DownSQL` in reverse migration order from current version down

to target. 3. Add test: `TestMigrationRollback` in `migrations_rollback_test.go` (already exists at path — verify it exists and covers Down). 4. **If down-migrations are NOT needed for production:** Document as “forward-only, restore from backup to rollback” — close G-19 honestly.

Verify:

```
ls server/internal/store/migrations_rollback_test.go
grep "DownSQL" server/internal/store/migrations.go | wc -l #
    should be 10
```

B-11 [AGENT] — HB-1 fw_setenv Flush-Error Handling

Effort: S (~1h, research-gated) **Source:** PRODUCTION_READINESS_PLAN.md K7

Current state: `server/internal/device/fwenv.go` `SaveEnv` swallows a `fw_setenv` flush error. A naive `return err false` fails on backends where empty-key-save is unsupported.

Fix: 1. Research `fw_setenv` no-args semantics (§11.4.8): does it error when there’s nothing to flush, or exit 0? 2. If exit 0 on no-op: add a pre-check — only call `fw_setenv` if there are actual env changes to flush. 3. If error on no-op: wrap the call, ignore specific “nothing to save” exit codes, propagate real errors. 4. Test: `TestFwSetenvNoOp`, `TestFwSetenvFlushError`.

B-12 [AGENT] — Webhook Dispatch for Deployment Lifecycle Events

Effort: M (~2h) **Source:** Gap tracker G-05

Current state: Webhook CRUD endpoints exist (`handlers_webhook.go`), webhook dispatch engine exists (`webhook.go`), but they may not be wired to deployment lifecycle events (deployment created → webhook fires; phase advanced → webhook fires; recall → webhook fires).

Fix: 1. In `handleCreateDeployment`, `handleEvaluateRollout`, `handleRecallDeployment` — call the webhook dispatcher after state change. 2. Ensure webhook payloads carry the event type, deployment

ID, release ID, and timestamp. 3. Test:
TestWebhookDispatchedOnDeploymentCreate,
TestWebhookDispatchedOnRolloutPhase.

B-13 [AGENT] — OTA Signing Metadata Crypto-Binding

Effort: M (~3h) **Source:** PRODUCTION_READINESS_PLAN.md K4, A-07 decision

If A-07 decides Option B (crypto-bind metadata): 1. Modify ota-artifact-validator to sign over SHA256(OSType || Board || Version || payloadDigest) instead of SHA256(payloadDigest) only. 2. Update scripts/testing/sign_artifact.go to include metadata in the signed digest. 3. Update server/internal/api/handlers_artifact.go verification to match. 4. Update Android agent VerifyBeforeApply to match. 5. This is a **breaking change** — requires coordinated deployment of server + all device agents. 6. Test: TestSignatureCoversMetadata, TestRelabelReplayBlocked.

If A-07 decides Option A (keep digest-only): Document the residual risk. TUF integration (ADR-0002, 1.0.1+) addresses this at a higher layer.

Verification Checklist

Step	Action	Expected Result
B-01	Verify SEC-1 fail-fast	Server refuses to boot with unset secret
B-02	Fix /readyz probe	503 on store failure
B-03	TLS pair validation	Error on one-of-pair
B-04	TLS in compose	podman-compose config passes
B-05	Rate-limit defaults	Server starts with limits active
B-06	Persistent auth	Users/refresh tokens survive restart
B-07	min_current_version wire	Device receives and enforces floor
B-08	Per-IP rate limiting	Login brute-force blocked

Step	Action	Expected Result
B-09	Security submodule wired	PII detection active
B-10	Migration Down()	Down-migrations runnable (or documented not-needed)
B-11	fw_setenv fix	Flush errors handled correctly
B-12	Webhook dispatch	Deployment events trigger webhooks
B-13	Signing metadata binding	Metadata included in signature (if decided)

Honest Boundary (§11.4.6)

Steps B-01 through B-13 are agent-executable. They do NOT require operator decisions (except B-05 which uses A-05's number, and B-13 which uses A-07's decision). All existing server code paths are referenced from static reads (`server/` directory, verified this session).